

The C Programming Language

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Introduction	2
2	Input / output	3
3	Working with files	8

Introduction

- **C++ features not in C:** This is clearly a non-exhaustive list, just the less obvious ones:
 1. Classes
 2. Member functions
 3. Constructors
 4. Destructors
 5. Access control: public, private, protected
 6. Inheritance
 7. Operator overloading
 8. Function overloading
 9. constexpr functions
 10. constexpr
 11. constexpr
 12. Namespaces
 13. References
 14. Move semantics
 15. RAII as a language-supported idiom
 16. Exceptions
 17. try / catch / throw
 18. new / delete
 19. Iterators
 20. Lambdas
 21. Capturing lambdas
 22. auto type deduction in the C++ sense
 23. decltype
 24. typeid
 25. dynamic_cast, static_cast, reinterpret_cast, const_cast
 26. Default function arguments
 27. Default member initializers
 28. Uniform initialization: initialization
 29. Member initializer lists
 30. this pointer
 31. Scoped enums: enum class
 32. bool as a built-in fundamental type in older C comparison
 33. References to functions/objects
 34. Inline variables
 35. Overload resolution
 36. Member pointers

Input / output

- **stdio.h:** In C, input/output is mostly handled through the standard I/O library, declared in:

```
0  #include <stdio.h>
```

- **Standard streams:** When a C program starts, it automatically has three standard streams:

```
0  stdin
1  stdout
2  stderr
```

- **printf:** Printf prints formatted text to stdout.

```
0  printf("%s %s", "Hello, ", "world!");
```

The placeholders are called format specifiers. Common ones include

```
0  %d      int
1  %u      unsigned int
2  %ld     long
3  %lu     unsigned long
4  %lld    long long
5  %llu    unsigned long long
6
7  %f      double
8  %lf     double in scanf, but also accepted in printf
9  %c      char
10 %s      string / char array
11 %p      pointer address
12 %%     literal percent sign
```

For %p, cast the pointer to void*. Note that the language itself does not make printf typesafe. Wrong format specifiers cause **undefined behavior**.

- **fprintf:** The fprintf function in C is used to write formatted data directly to an output stream, which is most commonly a file. It works identically to printf, except it requires a destination file pointer as its very first argument.

```
0  int fprintf(FILE *stream, const char *format, ...);
```

- **sprintf:** The sprintf function in C formats and stores a series of characters and values into a string buffer instead of printing them directly to the console. It is defined in the <stdio.h> standard library.

```
0  int sprintf(char *str, const char *format, ...);
```

Returns the total number of characters successfully written, excluding the null terminator (`\0`). If an error occurs, it returns a negative value.

- **snprintf:**

```
0  int snprintf ( char * s, size_t n, const char * format, ...  
    ↪ );
```

Composes a string with the same text that would be printed if `format` was used on `printf`, but instead of being printed, the content is stored as a C string in the buffer pointed by `s` (taking `n` as the maximum buffer capacity to fill).

If the resulting string would be longer than `n-1` characters, the remaining characters are discarded and not stored, but counted for the value returned by the function.

A terminating null character is automatically appended after the content written.

Returns the number of characters that would have been written if `n` had been sufficiently large, not counting the terminating null character. If an encoding error occurs, a negative number is returned.

The key rule is **size** is the total buffer size, including space for the null terminator.

```
0  char buf[10];  
1  snprintf(buf, sizeof buf, "hello");
```

`sizeof buf` is 10, so `snprintf` may write at most 9 visible characters plus the final `\0`.

```
0  char buf[6];  
1  snprintf(buf, sizeof buf, "hello");
```

This fits exactly. So, this is safe.

```
0  char buf[5];  
1  
2  int n = snprintf(buf, sizeof buf, "hello");
```

`snprintf` writes

```
0  'h' 'e' 'l' 'l' '\0'
```

in this case the return value is still 5, because "hello" would have required 5 visible characters, not counting the null terminator. `snprintf` returns the number of characters that would have been written, excluding the null terminator.

```
0 char buf[5];
1
2 int n = snprintf(buf, sizeof buf, "hello");
3
4 if (n < 0) {
5     // encoding or formatting error
6 } else if ((size_t)n >= sizeof buf) {
7     // output was truncated
8 } else {
9     // output fit
10 }
```

You can call `snprintf` once to find out how much space is needed:

```
0 int n = snprintf(NULL, 0, "value = %d", 123);
```

- **scanf**: `scanf` reads formatted input from `stdin`.

```
0 int x;
1 scanf("%d", &x)
```

`Scanf` needs the address because it has to modify the variable. `Scanf` returns the number of successful conversions. You should usually check the return value

```
0 int x;
1 if (scanf("%d", &x) != 1) {
2     fprintf(stderr, "Invalid integer\n");
3 }
```

- **puts**: Writes a string to the console

```
0 puts("Hello, world!");
```

- **getchar**: reads one character from `stdin`

```
0 int c;
1 c = getchar();
```

Notice that `getchar` returns `int`, not `char`. That is because it may return EOF, which is a special value indicating end-of-file or error.

```

0  int c;
1  while ((c=getchar()) != EOF) putchar(c);

```

- **putchar**: Writes one character to stdout

```

0  putchar('A');

```

- **fgets for line input**: Avoid

```

0  char name[32];
1  scanf("%s", name);

```

If the user types more than 31 characters, you can overflow the buffer. A safer approach is fgets:

```

0  char name[32];
1
2  printf("Enter your name: ");
3
4  if (fgets(name, sizeof name, stdin) == NULL) {
5      fprintf(stderr, "Input error\n");
6      return 1;
7  }
8
9  printf("Hello, %s", name);

```

Fgets has signature

```

0  char* fgets( char* str, int count, FILE* stream );

```

and returns **str** on success, null pointer on failure.

- **sscanf**: The sscanf function in C extracts and parses formatted data directly from a character string rather than reading from standard keyboard input (stdin). It is defined in the <stdio.h> library and stands for "String Scan Formatted".

```

0  int sscanf(const char *str, const char *format, ...);

```

```

0  const char *info = "ID:4023 Alice 28";
1  int id;
2  char name[20];
3  int age;
4
5  // Parse data by explicitly matching the literal "ID:" prefix
6  int items = sscanf(info, "ID:%d %19s %d", &id, name, &age);

```

- `%[set]`: A scanset. For instance,
- `%[set]`: *An inverted scanset. For instance,*
- *** Modifier: Suppresses assignment. For example,**

Working with files

- **Header:** File I/O is mainly done through the same header

```
0 #include <stdio.h>
```

- **FILE type:** The central type is

```
0 FILE*
```

A FILE* is a handle to an open file stream. You do not manipulate the file directly. You open it, get a FILE *, use functions such as fgetc, fgets, fprintf, fread, and finally close it with fclose.

- **fopen:**

```
0 FILE *fopen(const char *restrict filename, const char
  ↪ *restrict mode);
```

Upon successful completion fopen(), fdopen(), and freopen() return a FILE pointer. Otherwise, NULL is returned and errno is set to indicate the error.

The mode string tells C whether you want to read, write, append, or work in binary mode.

Mode	Meaning
"r"	Open existing file for reading
"w"	Open file for writing; destroys old contents
"a"	Open file for appending
"r+"	Open existing file for reading and writing
"w+"	Open for reading and writing; destroys old contents
"a+"	Open for reading and appending
"rb"	Read binary
"wb"	Write binary
"ab"	Append binary

The primary difference between r+ and a+ modes in C file handling lies in how they handle file creation and where the data is written. While both allow you to read and write without erasing (truncating) the existing data, they behave very differently under the hood.

Feature	r+ (Read / Update)	a+ (Append / Update)
File Must Exist?	Yes (returns NULL if missing)	No (creates a new file if missing)
Initial Pointer Position	Beginning of the file	End of the file
Overwrites Content?	Yes (overwrites at current cursor)	No (always forces writes to EOF)
Effect of fseek() on Writes	Moves the cursor for the next write	Ignored (writes always go to the end)

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     fclose(fp);
11     return 0;
12 }

```

- **fclose:**

```

0  int fclose(FILE* stream);

```

fclose closes an open file stream. It also flushes any buffered output before closing the file.

You should call fclose on every file you successfully opened with fopen.

Upon successful completion, 0 is returned. Otherwise, EOF is returned and errno is set to indicate the error.

EOF is a negative integer constant defined in <stdio.h>.

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("output.txt", "w");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     fprintf(fp, "Hello file\n");
11
12     if (fclose(fp) != EOF) {
13         perror("fclose");
14         return 1;
15     }
16
17     return 0;
18 }

```

- **The restrict keyword:** restrict is a type qualifier used on pointers to promise the compiler that the pointer is the only means to access its underlying memory object

When you qualify a pointer with restrict, you guarantee that for the lifetime of that pointer, no other pointer or variable inside the same scope will read or write to that specific chunk of memory.

- **fprintf:**

```

0  int fprintf(FILE *restrict stream, const char *restrict
    ↪  format, ...);

```

fprintf works like printf, but it writes to a file stream instead of standard output.

Returns the number of characters written on success.

Returns a negative value on failure.

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("numbers.txt", "w");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     int x = 10;
11     double y = 3.14;
12
13     fprintf(fp, "x = %d\n", x);
14     fprintf(fp, "y = %.2f\n", y);
15
16     fclose(fp);
17     return 0;
18 }

```

- **fputs:**

```

0  int fputs(const char *restrict s, FILE *restrict stream);

```

fputs writes a string to a file. Unlike puts, it does not automatically add a newline.

Returns a non-negative value on success. Returns EOF on failure.

- **fputc**

```

0  int fputc(int c, FILE *stream);

```

fputc writes a single character to a file. The character is passed as an int, but it usually represents a char.

Returns the character written on success. Returns EOF on failure.

- **fgetc:**

```

0  int fgetc(FILE *stream);

```

fgetc reads one character from a file.

It returns an int, not a char, because it needs to represent every possible character plus the special value EOF.

Returns the character read on success. Returns EOF if the end of the file is reached or if an error occurs.

```
0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     int ch;
11
12     while ((ch = fgetc(fp)) != EOF) {
13         putchar(ch);
14     }
15
16     fclose(fp);
17     return 0;
18 }
```

- **fgets:**

```
0  char *fgets(char *restrict s, int n, FILE *restrict stream);
```

fgets reads a line, or part of a line, from a file into a character buffer.

It reads at most $n - 1$ characters and then adds the null terminator `'\0'`.

If it sees a newline before the limit, it includes the newline in the buffer.

```

0  #include <stdio.h>
1
2  int main(void) {
3      FILE *fp = fopen("data.txt", "r");
4
5      if (fp == NULL) {
6          perror("fopen");
7          return 1;
8      }
9
10     char line[256];
11
12     while (fgets(line, sizeof line, fp) != NULL) {
13         printf("%s", line);
14     }
15
16     fclose(fp);
17     return 0;
18 }

```